

React Lab: ES6 Fundamentals — let, const, Classes, Arrow Functions, map() & Destructuring

Hands-on Lab Solution — "cricketapp" (ListofPlayers & IndianPlayers Components)

Objectives

- List the features of ES6
- Explain JavaScript let
- Identify the differences between var and let
- Explain JavaScript const
- Explain ES6 class fundamentals
- Explain ES6 class inheritance
- Define ES6 arrow functions
- Identify Set() and Map()

Lab Goals

- Use the map() method of ES6
- Apply arrow functions of ES6
- Implement destructuring features of ES6

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. Features of ES6 (ECMAScript 2015)

ES6 was a major update to JavaScript. Its key features include:

- let and const — block-scoped variable declarations
- Arrow functions — shorter function syntax with lexical 'this'
- Classes — syntactic sugar over JavaScript's prototype-based inheritance
- Template literals — string interpolation using backticks
- Destructuring assignment — unpacking values from arrays/objects into variables
- Default parameters — default values for function arguments
- Rest and spread operators (...) — gather or expand elements
- Modules — import / export syntax
- Promises — native support for asynchronous operations
- New data structures — Map and Set
- Array methods — map(), filter(), reduce(), find(), forEach(), etc.

2. JavaScript let

let declares a block-scoped variable, meaning it only exists within the nearest enclosing {} block (such as an if block, a for loop, or a function). Unlike var, a variable declared with let is not accessible before its declaration line (it is in a 'temporal dead zone') and cannot be re-declared in the same scope.

```
if (true) {  
  let score = 90;  
  console.log(score); // 90  
}  
console.log(score); // ReferenceError: score is not defined
```

3. Differences Between var and let

var	let
Function-scoped.	Block-scoped (limited to the nearest { }).
Can be re-declared in the same scope without error.	Cannot be re-declared in the same scope.
Hoisted and initialized with undefined, so it can be used before its declaration.	Hoisted but not initialized — accessing it before declaration throws a ReferenceError.
Creates a property on the global object (window) when declared globally.	Does not create a property on the global object.
Common source of bugs inside loops with closures.	Each loop iteration gets its own binding, avoiding common closure bugs.

4. JavaScript const

const also declares a block-scoped variable, but its binding cannot be reassigned after initialization, and it must be initialized at the time it is declared. Note that const only prevents reassignment of the variable itself — if the value is an array or object, its contents can still be mutated (e.g., pushing to a const array is allowed).

```
const PI = 3.14159;  
// PI = 3.14; // TypeError: Assignment to constant variable.  
  
const players = ['Rohit', 'Virat'];  
players.push('Rahul'); // allowed — the array contents can change
```

5. ES6 Class Fundamentals

ES6 classes provide a cleaner syntax for creating objects and dealing with inheritance, built on top of JavaScript's existing prototype system. A class typically has a constructor() method to initialize properties, and additional methods shared by every instance.

```
class Player {  
  constructor(name, score) {  
    this.name = name;
```

```

    this.score = score;
  }

  displayInfo() {
    return `${this.name} scored ${this.score} runs`;
  }
}

const p1 = new Player('Rohit Sharma', 85);
console.log(p1.displayInfo()); // Rohit Sharma scored 85 runs

```

6. ES6 Class Inheritance

A class can inherit from another class using the `extends` keyword. The child class can call the parent's constructor and methods using `super()`, and can override or add new methods.

```

class AllRounder extends Player {
  constructor(name, score, wickets) {
    super(name, score); // calls Player's constructor
    this.wickets = wickets;
  }

  displayInfo() {
    return `${super.displayInfo()} and took ${this.wickets} wickets`;
  }
}

const p2 = new AllRounder('Ravindra Jadeja', 72, 3);
console.log(p2.displayInfo());
// Ravindra Jadeja scored 72 runs and took 3 wickets

```

7. ES6 Arrow Functions

Arrow functions are a shorter syntax for writing functions, written as `(parameters) => expression`. They do not have their own `this` binding — they inherit this from the surrounding scope, which makes them especially convenient inside array methods like `map()` and `filter()`.

```

// Traditional function
function square(x) { return x * x; }

// Arrow function equivalent
const squareArrow = (x) => x * x;

// Used inside filter()
const lowScorers = players.filter((player) => player.score < 70);

```

8. Set() and Map()

ES6 introduced two new built-in collection types, in addition to the familiar `Array.prototype.map()` method:

- **Set** — a collection of unique values (no duplicates). Useful for de-duplicating data, e.g. `new Set([1, 2, 2, 3])` results in `{1, 2, 3}`.
- **Map** — a collection of key-value pairs where keys can be of any type (unlike plain objects, which coerce keys to strings).

- `Array.prototype.map(callback)` — an array method (used heavily in this lab) that creates a new array by applying a function to every element of the original array, without mutating the original.

```
// Set: removes duplicate scores
const uniqueScores = new Set([85, 92, 85, 45, 92]);
console.log(uniqueScores); // Set(3) {85, 92, 45}

// Map: key-value collection
const teamMap = new Map();
teamMap.set('captain', 'Rohit Sharma');
teamMap.set('viceCaptain', 'Shubman Gill');
console.log(teamMap.get('captain')); // Rohit Sharma

// Array.prototype.map(): transforms every array element
const names = players.map((player) => player.name);
```

Lab Steps

Step 1: Create the React application

```
npx create-react-app cricketapp
cd cricketapp
npm start
```

Open the cricketapp folder in Visual Studio Code. Inside the src folder, create two new files: `ListofPlayers.js` and `IndianPlayers.js`.

Step 2: Create the `ListofPlayers` component

This component declares an array of 11 players (name and score), renders the full list using the ES6 `map()` method, and then uses `filter()` with an arrow function to show only the players who scored below 70.

src/ListofPlayers.js

```
import React from 'react';

function ListofPlayers() {
  // ES6 array of 11 player objects
  const players = [
    { name: 'Rohit Sharma',    score: 85 },
    { name: 'Virat Kohli',     score: 92 },
    { name: 'KL Rahul',        score: 45 },
    { name: 'Shubman Gill',     score: 78 },
    { name: 'Shreyas Iyer',     score: 55 },
    { name: 'Rishabh Pant',     score: 60 },
    { name: 'Hardik Pandya',    score: 40 },
    { name: 'Ravindra Jadeja',  score: 72 },
    { name: 'Mohammed Shami',   score: 20 },
    { name: 'Jasprit Bumrah',   score: 15 },
    { name: 'Kuldeep Yadav',    score: 30 }
  ];

  // Arrow function + filter(): players who scored below 70
  const lowScorers = players.filter((player) => player.score < 70);

  return (
    <div>
```

```

    <h2>List of Players</h2>
    <ul>
      { /* map(): render every player */ }
      { players.map((player, index) => (
        <li key={index}>{player.name} - {player.score}</li>
      )) }
    </ul>

    <h3>Players with Score Below 70</h3>
    <ul>
      { lowScorers.map((player, index) => (
        <li key={index}>{player.name} - {player.score}</li>
      )) }
    </ul>
  </div>
);
}

export default ListofPlayers;

```

Step 3: Create the IndianPlayers component

Part (a) uses ES6 array destructuring to split the 11 players into an Odd Team (players at positions 1, 3, 5 ...) and an Even Team (players at positions 2, 4, 6 ...). Part (b) declares two separate arrays, T20players and RanjiTrophyPlayers, and merges them into one array using the ES6 spread operator (...) — the ES6 'merge' feature for arrays.

src/IndianPlayers.js

```

import React from 'react';

function IndianPlayers() {
  const players = [
    'Rohit Sharma', 'Virat Kohli', 'KL Rahul', 'Shubman Gill',
    'Shreyas Iyer', 'Rishabh Pant', 'Hardik Pandya', 'Ravindra Jadeja',
    'Mohammed Shami', 'Jasprit Bumrah', 'Kuldeep Yadav'
  ];

  // (a) Destructuring: pull all 11 names out into individual variables
  const [p1, p2, p3, p4, p5, p6, p7, p8, p9, p10, p11] = players;

  const oddTeamPlayers = [p1, p3, p5, p7, p9, p11]; // positions 1,3,5...
  const evenTeamPlayers = [p2, p4, p6, p8, p10]; // positions 2,4,6...

  // (b) Two separate arrays
  const T20players = ['Rohit Sharma', 'Virat Kohli', 'Suryakumar Yadav'];
  const RanjiTrophyPlayers = ['Sarfaraz Khan', 'Yashasvi Jaiswal', 'Prithvi Shaw'];

  // Merge feature: ES6 spread operator combines both arrays
  const mergedPlayers = [...T20players, ...RanjiTrophyPlayers];

  return (
    <div>
      <h2>Indian Players</h2>

      <h3>Odd Team Players</h3>
      <ul>
        { oddTeamPlayers.map((name, index) => <li key={index}>{name}</li>)}
      </ul>
    </div>
  );
}

```

```

    </ul>

    <h3>Even Team Players</h3>
    <ul>
      {evenTeamPlayers.map((name, index) => <li key={index}>{name}</li>)}
    </ul>

    <h3>Merged Players (T20 + Ranji Trophy)</h3>
    <ul>
      {mergedPlayers.map((name, index) => <li key={index}>{name}</li>)}
    </ul>
  </div>
);
}

export default IndianPlayers;

```

Step 4: Show both components on the home page using a flag

App.js declares a flag variable and uses a simple if / else to decide what to render: when flag is true, both ListofPlayers and IndianPlayers are displayed; when flag is false, an alternate message is shown instead.

src/App.js

```

import React from 'react';
import ListofPlayers from './ListofPlayers';
import IndianPlayers from './IndianPlayers';

function App() {
  const flag = true; // change to false to see the else branch

  let content;
  if (flag) {
    content = (
      <div>
        <ListofPlayers />
        <IndianPlayers />
      </div>
    );
  } else {
    content = <h2>No Data to Display</h2>;
  }

  return (
    <div className="App">
      <h1>Cricket App</h1>
      {content}
    </div>
  );
}

export default App;

```

Step 5: Render with ReactDOM

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 6: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>.

Expected Output

When flag = true

```
Cricket App

List of Players
Rohit Sharma - 85    Virat Kohli - 92    KL Rahul - 45
Shubman Gill - 78    Shreyas Iyer - 55    Rishabh Pant - 60
Hardik Pandya - 40    Ravindra Jadeja - 72    Mohammed Shami - 20
Jasprit Bumrah - 15    Kuldeep Yadav - 30

Players with Score Below 70
KL Rahul - 45    Shreyas Iyer - 55    Rishabh Pant - 60
Hardik Pandya - 40    Mohammed Shami - 20    Jasprit Bumrah - 15
Kuldeep Yadav - 30

Indian Players
Odd Team Players: Rohit Sharma, KL Rahul, Shreyas Iyer,
                  Hardik Pandya, Mohammed Shami, Kuldeep Yadav
Even Team Players: Virat Kohli, Shubman Gill, Rishabh Pant,
                  Ravindra Jadeja, Jasprit Bumrah
Merged Players (T20 + Ranji Trophy): Rohit Sharma, Virat Kohli,
                  Suryakumar Yadav, Sarfaraz Khan, Yashasvi Jaiswal, Prithvi Shaw
```

When flag = false

```
Cricket App

No Data to Display
```

Summary

- let and const are block-scoped; let allows reassignment, const does not allow the binding to be reassigned.
- var is function-scoped and hoisted with an initial value of undefined, unlike let/const.
- ES6 classes (class ... extends ... { constructor() { super(); } }) give a clean syntax for objects and inheritance.
- Arrow functions provide concise syntax and are ideal for use inside map()/filter().

- `Array.prototype.map()` transforms every element of an array into a new array (used here to render lists in JSX).
- Set stores unique values and Map stores key-value pairs — both are new ES6 collection types.
- Destructuring unpacks array/object values into individual variables in one line.
- The spread operator (`...`) merges arrays, as used to combine `T20players` and `RanjiTrophyPlayers`.
- A simple `if / else` on a flag variable controls which components are rendered on the home page.